

# ASSIGNMENT – 1

Working with Python packages – NumPy, Pandas, Scikit-Learn, Matplotlib

## AIM

To explore and gain practical experience with fundamental Python libraries used in Machine Learning – NumPy, Pandas, Scikit-learn and Matplotlib – and to perform data preprocessing and exploratory data analysis (EDA) on the Diabetes Prediction dataset, covering missing value handling, irrelevant feature removal, categorical encoding, feature scaling, correlation and mutual information analysis, visualization, and dataset splitting for model development.

## CODE

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg") # no GUI windows
import matplotlib.pyplot as plt

# Load dataset

df = pd.read_csv("Diabetes_prediction(2).csv")
print("Dataset loaded successfully\n")

print("First Five Records")
print(df.head())

print("\nNumber of Rows")
print(df.shape[0])

print("\nNumber of Columns")
print(df.shape[1])

print("\nFeature Names")
print(df.columns.tolist())

# Identify target column

target = "Diagnosis"
print("\nTarget Column :", target)

# Missing values

print("\nMissing Values")
print(df.isnull().sum())

df = df.fillna(df.mean(numeric_only=True))

print("\nMissing Values After Handling")
print(df.isnull().sum())

# Remove irrelevant features
```

```
if "id" in df.columns:
    df.drop("id", axis=1, inplace=True)
    print("\nID column removed")
else:
    print("\nNo irrelevant feature found")

# Remove duplicates

print("\nDuplicates Before Removal :", df.duplicated().sum())
df = df.drop_duplicates()
print("Duplicates After Removal :", df.duplicated().sum())

# Encode categorical variables

from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
categorical = df.select_dtypes(include="object").columns

for col in categorical:
    df[col] = encoder.fit_transform(df[col])

print("\nCategorical Features Encoded")

# Standardize numerical input features

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

numerical_features = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age"
]
```

```
df[numerical_features] = scaler.fit_transform(df[numerical_features])

print("\nNumerical Features Standardized")
print(df.head())

# Correlation with target

correlation = df.corr(numeric_only=True)

print("\nCorrelation with Target")
print(correlation[target].sort_values(ascending=False))

# Mutual information

from sklearn.feature_selection import mutual_info_classif

X = df.drop(target, axis=1)
y = df[target]

mi = mutual_info_classif(X, y)
mi_scores = pd.Series(mi, index=X.columns)
mi_scores = mi_scores.sort_values(ascending=False)

print("\nMutual Information Scores")
print(mi_scores)

# Feature importance table

importance = pd.DataFrame({
    "Feature": X.columns,
    "Correlation": correlation[target].loc[X.columns],
    "Mutual Information": mi_scores.loc[X.columns]
}).sort_values(by="Mutual Information", ascending=False)

print("\nFeature Importance")
print(importance)

# EDA visualizations
```

```
print("\nSummary Statistics")
print(df.describe())

# Histograms

df.hist(figsize=(14, 10))
plt.suptitle("Histogram of Features")
plt.show()

# Bar chart of target distribution

class_count = df[target].value_counts()

plt.figure(figsize=(6, 5))
plt.bar(class_count.index.astype(str), class_count.values, color=["skyblue", "orange"])
plt.xlabel("Diagnosis")
plt.ylabel("Count")
plt.title("Target Distribution")
plt.show()

# Scatter plot: Glucose vs BMI

plt.figure(figsize=(7, 5))

diabetic = df[df[target] == 1]
non_diabetic = df[df[target] == 0]

plt.scatter(non_diabetic["Glucose"], non_diabetic["BMI"], color="blue", label="Non-Diabetic")
plt.scatter(diabetic["Glucose"], diabetic["BMI"], color="red", label="Diabetic")

plt.xlabel("Glucose")
plt.ylabel("BMI")
plt.title("Glucose vs BMI")
plt.legend()
plt.show()

# Box plot

plt.figure(figsize=(12, 6))
numeric_columns = df.select_dtypes(include=np.number).columns
```

```
plt.boxplot([df[col] for col in numeric_columns], tick_labels=numeric_columns)
plt.xticks(rotation=45)
plt.title("Box Plot of Numerical Features")
plt.show()

# Heatmap

corr = df.corr(numeric_only=True)

plt.figure(figsize=(10, 8))
plt.imshow(corr, cmap="coolwarm")
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title("Correlation Heatmap")
plt.show()

# Pair plot

plt.figure(figsize=(6, 5))
plt.scatter(df["Glucose"], df["BMI"])
plt.xlabel("Glucose")
plt.ylabel("BMI")
plt.title("Glucose vs BMI")
plt.show()

# Class balance

print("\nClass Distribution")

class_count = df[target].value_counts()

plt.figure(figsize=(6, 5))
plt.bar(class_count.index.astype(str), class_count.values, color=["green", "red"])
plt.xlabel("Diagnosis")
plt.ylabel("Number of Samples")
plt.title("Class Distribution")
plt.show()

# Train / validation / test split
```

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42)
X_validation, X_test, y_validation, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42)

print("\nTraining Shape")
print(X_train.shape)

print("\nValidation Shape")
print(X_validation.shape)

print("\nTesting Shape")
print(X_test.shape)
```

## OUTPUT

Dataset loaded successfully

First Five Records

|   | Pregnancies | Glucose    | ... | Age       | Diagnosis |
|---|-------------|------------|-----|-----------|-----------|
| 0 | 2           | 115.863387 | ... | 20.100494 | 0         |
| 1 | 2           | 92.490122  | ... | 44.912281 | 0         |
| 2 | 1           | 88.141469  | ... | 48.247873 | 1         |
| 3 | 2           | 108.453101 | ... | 42.749868 | 0         |
| 4 | 1           | 127.849443 | ... | 32.797789 | 0         |

[5 rows x 9 columns]

Number of Rows

1000

Number of Columns

9

Feature Names

['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Diagnosis']

Target Column : Diagnosis

Missing Values

|                          |       |
|--------------------------|-------|
| Pregnancies              | 0     |
| Glucose                  | 0     |
| BloodPressure            | 0     |
| SkinThickness            | 0     |
| Insulin                  | 0     |
| BMI                      | 0     |
| DiabetesPedigreeFunction | 0     |
| Age                      | 0     |
| Diagnosis                | 0     |
| dtype:                   | int64 |

Missing Values After Handling

```
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI               0
DiabetesPedigreeFunction  0
Age               0
Diagnosis         0
dtype: int64
```

No irrelevant feature found

Duplicates Before Removal : 0

Duplicates After Removal : 0

Categorical Features Encoded

Numerical Features Standardized

|   | Pregnancies | Glucose   | ... | Age       | Diagnosis |
|---|-------------|-----------|-----|-----------|-----------|
| 0 | 0.169163    | 0.843882  | ... | -1.603337 | 0         |
| 1 | 0.169163    | -0.357150 | ... | 0.112773  | 0         |
| 2 | -0.569542   | -0.580604 | ... | 0.343479  | 1         |
| 3 | 0.169163    | 0.463106  | ... | -0.036791 | 0         |
| 4 | -0.569542   | 1.459784  | ... | -0.725127 | 0         |

[5 rows x 9 columns]

Correlation with Target

|                          |           |
|--------------------------|-----------|
| Diagnosis                | 1.000000  |
| DiabetesPedigreeFunction | 0.060549  |
| Insulin                  | 0.035411  |
| BloodPressure            | 0.010349  |
| SkinThickness            | -0.000014 |

```
Pregnancies      -0.001484
Glucose          -0.004971
BMI              -0.033336
Age              -0.044726
Name: Diagnosis, dtype: float64
```

#### Mutual Information Scores

```
DiabetesPedigreeFunction  0.008826
Insulin                  0.002899
Age                      0.002413
BMI                      0.001744
SkinThickness            0.000000
BloodPressure            0.000000
Glucose                  0.000000
Pregnancies              0.000000
dtype: float64
```

#### Feature Importance

|                          | Feature                  | ... | Mutual Information |
|--------------------------|--------------------------|-----|--------------------|
| DiabetesPedigreeFunction | DiabetesPedigreeFunction | ... | 0.008826           |
| Insulin                  | Insulin                  | ... | 0.002899           |
| Age                      | Age                      | ... | 0.002413           |
| BMI                      | BMI                      | ... | 0.001744           |
| SkinThickness            | SkinThickness            | ... | 0.000000           |
| BloodPressure            | BloodPressure            | ... | 0.000000           |
| Glucose                  | Glucose                  | ... | 0.000000           |
| Pregnancies              | Pregnancies              | ... | 0.000000           |

[8 rows x 3 columns]

#### Summary Statistics

|       | Pregnancies  | Glucose       | ... | Age           | Diagnosis  |
|-------|--------------|---------------|-----|---------------|------------|
| count | 1.000000e+03 | 1.000000e+03  | ... | 1.000000e+03  | 1000.00000 |
| mean  | 1.136868e-16 | -3.090861e-16 | ... | -1.065814e-17 | 0.30600    |
| std   | 1.000500e+00 | 1.000500e+00  | ... | 1.000500e+00  | 0.46106    |

```
min    -1.308246e+00 -3.538833e+00 ... -3.061357e+00    0.00000
25%    -5.695415e-01 -6.831450e-01 ... -6.752827e-01    0.00000
50%     1.691634e-01  9.123268e-04 ...  2.437897e-02     0.00000
75%     9.078684e-01  7.103414e-01 ...  6.789692e-01     1.00000
max     4.601393e+00  3.175497e+00 ...  3.270954e+00     1.00000
```

```
[8 rows x 9 columns]
```

```
[captured plot 1 -> plot_01.png]
```

```
[captured plot 2 -> plot_02.png]
```

```
[captured plot 3 -> plot_03.png]
```

```
[captured plot 4 -> plot_04.png]
```

```
[captured plot 5 -> plot_05.png]
```

```
[captured plot 6 -> plot_06.png]
```

Class Distribution

```
[captured plot 7 -> plot_07.png]
```

Training Shape

```
(700, 8)
```

Validation Shape

```
(150, 8)
```

Testing Shape

```
(150, 8)
```

## PLOTS

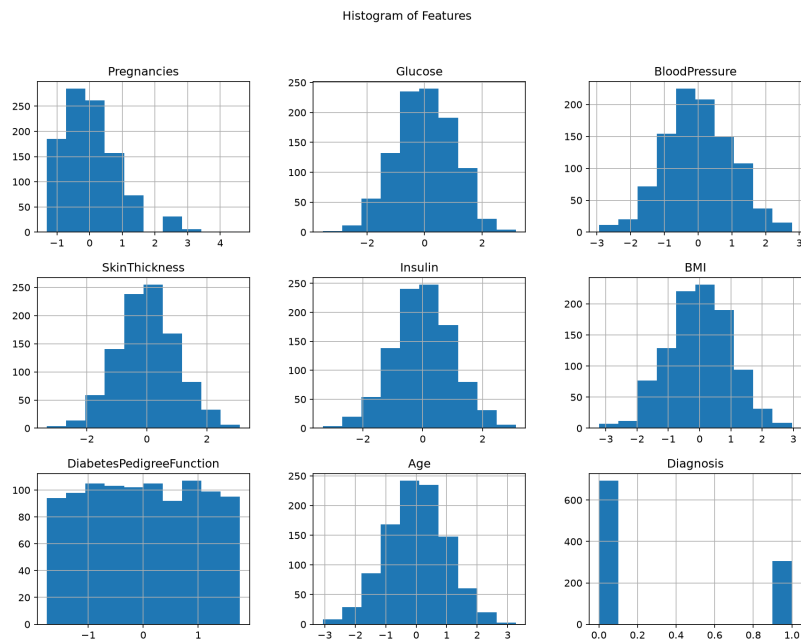


Fig 1 – Histogram of Features

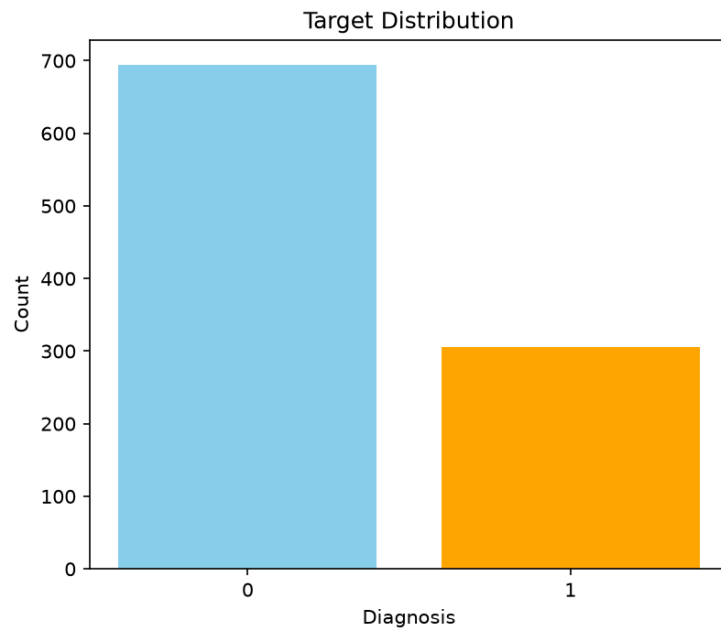


Fig 2 - Target Distribution

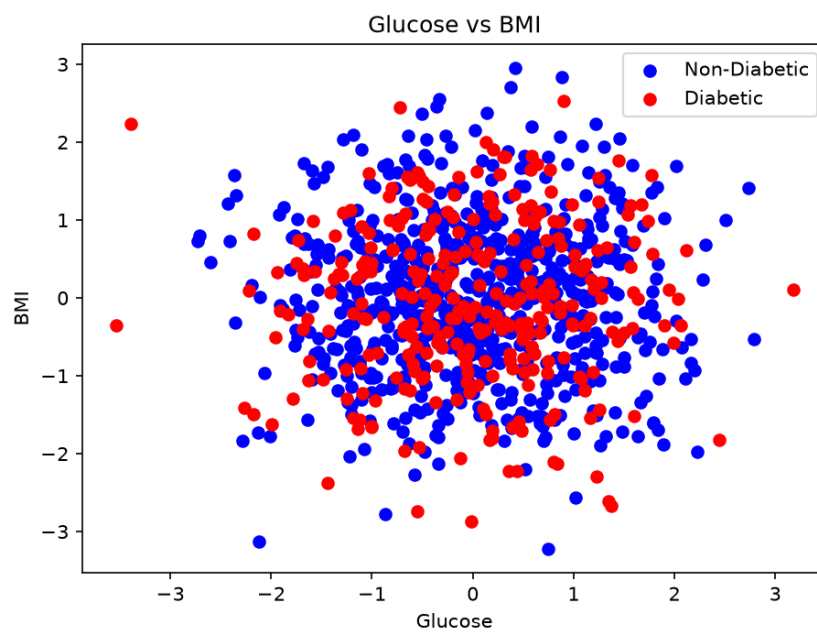


Fig 3 - Glucose vs BMI Scatter Plot

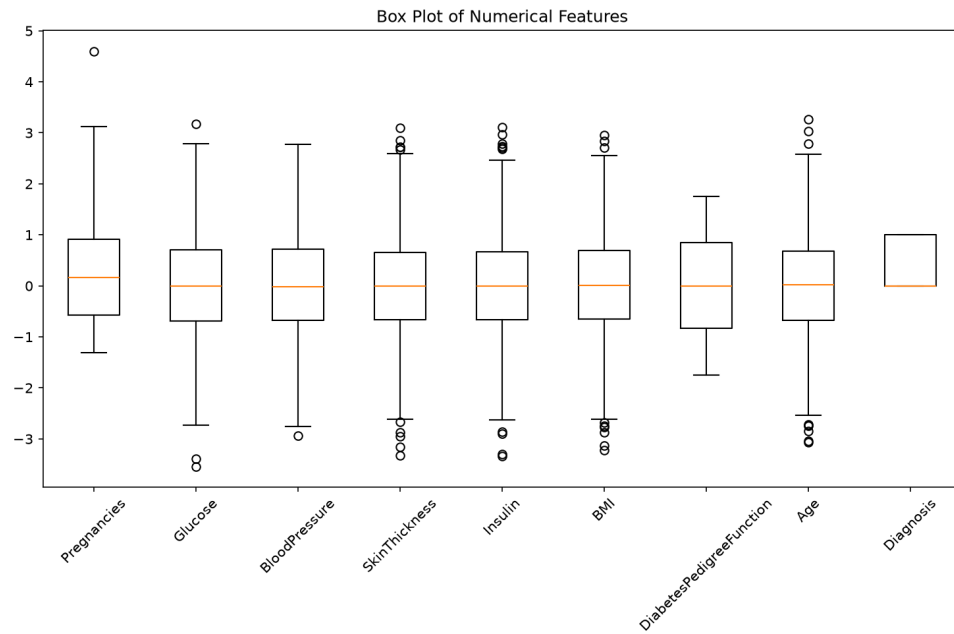


Fig 4 - Box Plot of Numerical Features

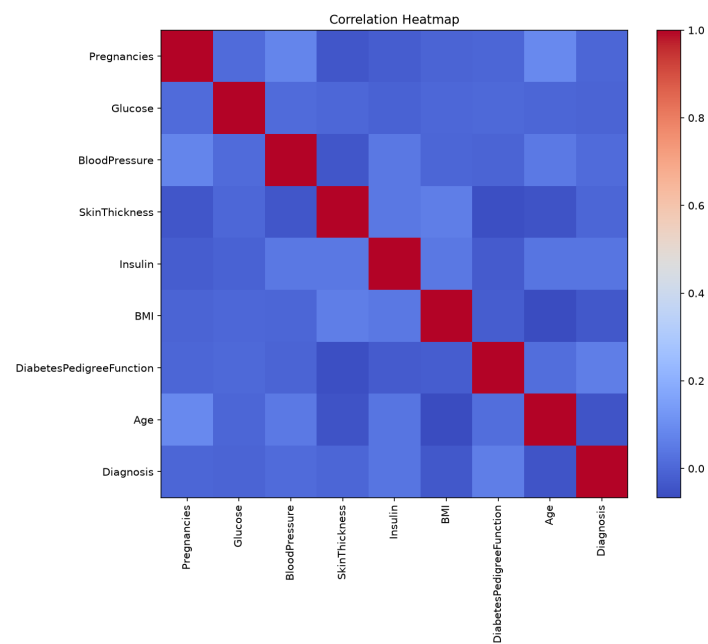
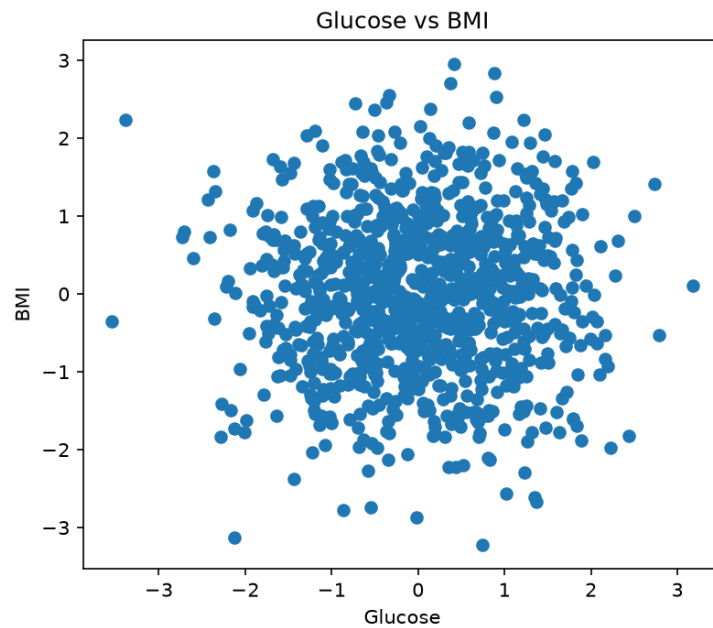
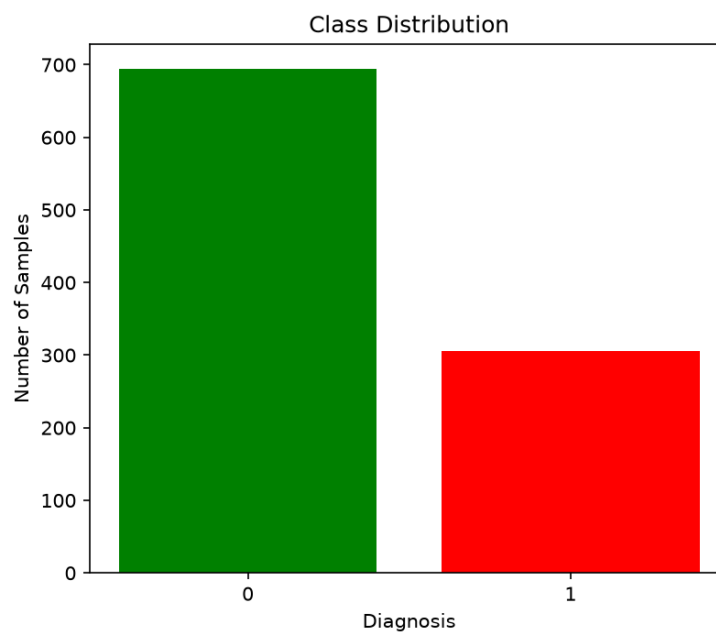


Fig 5 - Correlation Heatmap

*Fig 6 - Pair Plot (Glucose vs BMI)**Fig 7 - Class Distribution*

## TASKS PERFORMED

|                     | Method / Description                                                    | Result                                              |
|---------------------|-------------------------------------------------------------------------|-----------------------------------------------------|
|                     | <code>pandas read_csv</code> on <code>Diabetes_prediction(2).csv</code> | 1000 rows x 9 columns                               |
|                     | Display first five records and column names                             | 8 input features, target = Diagnosis                |
| Missing values      | <code>fillna</code> with column mean for numerical columns              | 0 missing values after handling                     |
| Irrelevant features | Drop <code>id</code> column if it exists                                | No <code>id</code> column found; no feature removed |
| Duplicates          | <code>df.drop_duplicates()</code>                                       | 0 duplicates before and after                       |

|                | Method / Description                                                        | Result                                 |
|----------------|-----------------------------------------------------------------------------|----------------------------------------|
| ical variables | LabelEncoder on object-type columns                                         | No categorical columns to encode       |
|                | StandardScaler on all 8 numerical input features                            | Features standardized to zero mean     |
| ysis           | df.corr() with Diagnosis                                                    | DiabetesPedigreeFunction highest       |
| ion            | mutual_info_classif(X, y)                                                   | DiabetesPedigreeFunction highest       |
| nce table      | Tabulate correlation and mutual information                                 | Top feature: DiabetesPedigreeFun       |
| ics            | df.describe()                                                               | Mean values close to 0, std close to   |
|                | Histograms, bar chart, scatter, box plot, heatmap, pair plot, class balance | 7 plots generated                      |
| /test split    | train_test_split with test_size=0.30 then 0.50, random_state=42             | (700,8) train, (150,8) validation, (15 |

## LEARNING OUTCOMES

- I was able to load and inspect a real dataset using pandas and identify its shape, features and target column.
- I understood how to handle missing values by replacing them with the mean of each numerical column.
- I learnt how to remove duplicate records and irrelevant features like an ID column.
- I applied StandardScaler to standardize numerical input features before analysis.
- I computed correlation and mutual information scores to find which features relate most to the target.
- I created different visualizations to explore the distribution, spread and relationship between features.
- I split the dataset into training, validation and testing sets in a 70:15:15 ratio for model development.